SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

arXiv:2310.06770

Feliciann Elliot

MAI 5301 - Foundations Of Large Language Models

Abstract

Current language models are advancing faster than our ability to evaluate them. To better assess real-world capabilities, the authors introduce SWE-bench, a benchmark based on over 2,000 real GitHub software issues across multiple Python repositories. In this task, models must modify an existing codebase to resolve documented issues, often requiring reasoning across multiple files and complex system interactions. Results show that even state-of-the-art models struggle with these tasks, solving only a small fraction of issues. This highlights the gap between current model capabilities and the requirements of real-world software engineering.

Introduction

Language models are increasingly used in real-world tools such as chatbots and coding assistants, yet many existing benchmarks no longer capture the limits of modern models. While coding benchmarks exist, most focus on small, self-contained tasks that do not reflect real software engineering challenges. In practice, fixing software issues often requires understanding large codebases and coordinating changes across multiple files. To address this gap, the authors introduce SWE-bench, a benchmark where models must resolve real GitHub issues by generating patches to modify existing repositories. The benchmark provides realistic tasks, automated evaluation through repository test suites, and continuously expandable data. Experiments show that even advanced language models solve only a small portion of these problems, highlighting the difficulty of real-world software engineering for current LMs.

What Problem Is This Paper Solving?

Current LLM benchmarks like HumanEval are saturated and evaluate narrow, self-contained problems solvable in a few lines of code. They don't reflect what real software engineering actually demands.

- Real bugs require navigating thousands of files, understanding cross-function dependencies, and reasoning about how a change in one module ripples through others
- Existing benchmarks pigeonhole models into a reduced role and do not let models demonstrate versatility or expose genuine capability gaps

There's a pressing need for a benchmark that is challenging, sustainable, and continuously updatable to track the true frontier of LLM capability

What SWE-bench Is

SWE-bench is an evaluation framework consisting of 2,294 software engineering problems drawn from real GitHub issues and pull requests across 12 popular Python repositories.

- Given a codebase + issue description, the model must edit the codebase to resolve the issue
- Solutions evaluated by running the repository's actual unit and system tests no hand-crafted checkers
- Reference solutions average editing 1.7 files, 3.0 functions, and 32.8 lines per task

Figure 1: SWE-bench sources task instances from real-world Python repositories by connecting GitHub issues to merged pull request solutions that resolve related tests. Provided with the issue text and a codebase snapshot, models generate a patch that is evaluated against real tests.

Diagram showing the pipeline from GitHub issue → codebase → LM → generated PR → test evaluation.

What SWE-bench is

- Evaluations run at least one fail-to-pass test plus a median of 51 others for regression.
- Without hints on where to edit, it forces creative, cross-context solutions amid vast inputs.

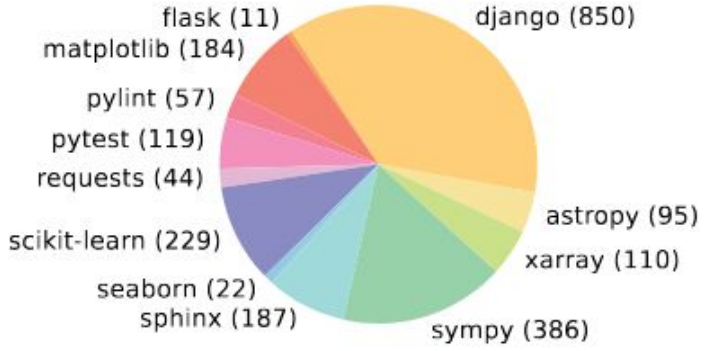

Figure 3: Distribution of SWE-bench tasks (in parenthesis) across 12 open source GitHub repositories that each contains the source code for a popular, widely downloaded PyPI package.

Key Terms

- Patch Generation - The main task in SWE-bench.
  Models must generate a patch (diff-style code changes) that modifies the existing codebase to resolve a GitHub issue. The patch must apply successfully and pass the repository's test suite.

- Gold Patch (Reference Solution)
  The human-written patch from the original merged pull request that correctly fixes the issue. It serves as the ground truth used to evaluate whether a model's generated patch achieves the same functional outcome.

Key Terms

- No-Op Patch (No Operation)
  A patch that applies successfully but does not meaningfully change behavior. Examples include formatting changes, comments, or trivial edits. This is a common failure case where the model makes no useful fix.

- Regression
  A generated patch that introduces new bugs or breaks existing functionality. Even if the target issue is fixed, failing unrelated tests makes the patch invalid.

- Fail-to-Pass Test
  A test that fails on the original buggy code but passes after the correct patch is applied. SWE-bench requires at least one such test per instance to verify that the issue is truly fixed.

SWE-Llama – Open Model Attempt

Built on CodeLlama's 7B and 13B variants, which struggled out-of-the-box with long contexts and instructions, SWE-Llama gets fine-tuned on 19,000 issue-PR pairs from separate repos.

- Using LoRA keeps it efficient while enabling over 100,000-token handling and proper patch generation.
- This specialization makes open models viable competitors to proprietary ones in targeted editing.

SWE-Llama – Impact & Limitations

Under realistic BM25 retrieval, the model resolves only 0.70% of tasks, matching the performance of the 7B version.

- Most patches apply cleanly but still fail tests because the model cannot coordinate changes across multiple files or avoid regressions.
- This gap shows fine-tuning alone is not enough.
- We now need to see how even the strongest proprietary models perform in the full experimental setup.

How Task Instances Are Constructed (The Pipeline)

The construction pipeline is a 3-stage process designed to ensure quality and executability at scale, starting from ~90,000 raw PRs filtered down to 2,294.

- Stage 1 — Scraping: PRs collected from 12 popular Python repos (chosen for documentation quality, test coverage, and contributor guidelines)
- Stage 2 — Attribute filtering: Keep only merged PRs that (a) resolve a GitHub issue and (b) introduce new tests, ensuring the fix is verified
- Stage 3 — Execution filtering: Each candidate is installed and run; only instances with at least one test that changes from fail to pass are kept

Figure 2: SWE-bench task instances are created from merged pull requests that resolve an issue, contributes tests, and install successfully.

The three-stage pipeline diagram (Scrape PRs → Attribute Filter → Execution Filter with tick criteria)

Experimental Setup: Retrieval Problem

A fundamental challenge before even running a model is that codebases are far too large to fit in any context window.

- An average codebase of 438K lines far exceeds even the largest context windows available

Two retrieval strategies are tested:

- BM25 sparse retrieval: a generic keyword-based retriever that selects the most relevant files to insert as context; three context window sizes tested (13k, 27k, 50k tokens)
- "Oracle" retrieval: for analysis only — provides the exact files edited by the reference solution (not realistic in practice, since you wouldn't know which files to edit before solving the issue)

Experimental Setup: Retrieval Problem

- BM25 only retrieves a superset of oracle files in approximately 40% of cases at 27k tokens; in nearly half of instances, it retrieves none of the correct files

- This retrieval bottleneck is itself a significant finding as it demonstrates that model performance is heavily bounded by how well you can identify the relevant code before the model even begins reasoning.

The Results

- The best-performing model, Claude 2, resolves only 1.96% of issues under BM25 retrieval.
- Claude 3 Opus improves this to 3.79%, but the ceiling remains extremely low
- ChatGPT-3.5 resolves just 0.17%, nearly nothing
- Even with oracle retrieval (an unrealistic upper bound), Claude 2 only reaches 4.8%

When context is collapsed to only the exact lines needing editing ±15 lines, performance improves to 5.93%, showing models can reason correctly when given the right context, but finding that context is the unsolved problem

The Results

Table 5: We compare models against each other using the BM25 retriever as described in Section 4.

	SWE-bench		SWE-bench Lite	
Model	% Resolved	% Apply	% Resolved	% Apply
Claude 3 Opus	**3.79**	46.56	**4.33**	**51.67**
Claude 2	1.97	43.07	3.00	33.00
ChatGPT-3.5	0.17	26.33	0.33	10.00
GPT-4-turbo	1.31	26.90	2.67	29.67
SWE-Llama 7b	0.70	51.74	1.33	38.00
SWE-Llama 13b	0.70	**53.62**	1.00	38.00

- The retrieval bottleneck severely limits performance: BM25 only retrieves a superset of the correct files in ~40% of cases (at 27k tokens), and in nearly half of instances retrieves none of them. Model capability is heavily constrained by the ability to identify relevant code before reasoning even begins.

Why Performance Degrades with More Context

Counter-intuitively, giving models more context makes them perform worse.

Performance drops significantly as total input tokens increase. The models become distracted by irrelevant code.

This corroborates "lost in the middle" (Liu et al., 2023b) findings in the broader literature which notes that LLMs struggle to localize target information when surrounded by noise.

Why Performance Degrades with More Context

Table 2 shows how Claude 2 performs best at the shortest context window (13k), not the largest (50k)

Table 2: Model resolve rates with BM25 retrieval, with different maximum context lengths.

	Max. Content		
Model	13k	27k	50k
Claude 2	**1.96**	**1.87**	**1.22**
SWE-Llama 7b	0.70	0.31	0.00
SWE-Llama 13b	0.70	0.48	0.00

Why Performance Degrades with More Context

The implication is that raw context length is not the answer.

Models need architectural support for intelligent, selective attention to the environment, not just bigger windows.

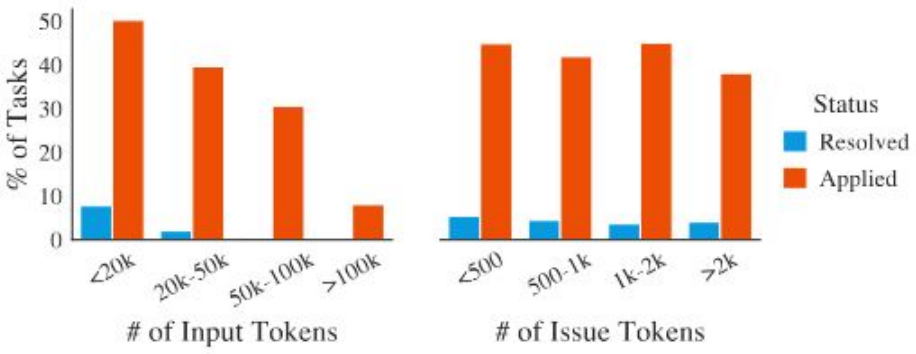

Figure 5: We compare the performance of Claude 2 on tasks partitioned by total input length and by only the issue length.

Qualitative Failure Analysis

Even when patches apply successfully, most model generations still fail the tests. The paper's detailed analysis of SWE-Llama and Claude 2 generations reveals consistent failure patterns:

- Models write primitive Python by ignoring existing helper functions, abstractions, and third-party libraries already in the codebase.
- They take a greedy, minimal-fix approach of solving only the immediate symptom while ignoring code style, logical constraints, and future maintainability.

Qualitative Failure Analysis

- When patches do apply, the majority are No-Ops (correct formatting, wrong logic) or cause Regressions (break existing tests)

Model	Claude 2	ChatGPT-3.5	GPT-4*	SWE-Llama 7b	SWE-Llama 13b
Applied	1078	284	76	1257	1196
Resolved	110	12	10	69	91
Breaking Resolved	26	2	3	17	10
Partially Resolved	15	4	3	17	10
Work in Progress	20	2	1	17	16
No-Op	471	174	30	716	672
Regression	436	90	29	421	397

Table 23: Categorization of model generations that applied successfully by the cases defined in Table 22. As mentioned, GPT-4 was evaluated on a 25% subset (574 instances) of SWE-bench.

Qualitative Failure Analysis

- Gold (human) patches often make structural improvements that prevent future bugs; model patches almost never do.

Table 8: Average edits of model generated patches in the "oracle" retrieval setting across successfully applied patches. For the task instances specific to each model, we calculate the same statistics across the gold patches. Avg Gold shows statistics macro-averaged over each models' respective gold patches. All Gold shows statistics for all gold patches unconditioned on model performance.

Model	Total Lines	Added	Removed	Functions	Files
Claude 2	19.6	4.2	1.9	1.1	1.0
Gold	44.1	12.0	5.8	2.1	1.2
ChatGPT-3.5	30.1	3.8	2.7	1.6	1.0
Gold	39.6	9.5	6.1	1.9	1.2
GPT-4	20.9	4.4	1.5	1.0	1.0
Gold	33.6	8.4	3.8	1.9	1.1
SWE-Llama 13b	17.6	1.6	1.2	1.2	1.1
Gold	37.8	10.0	4.4	1.9	1.1
SWE-Llama 7b	16.7	1.3	1.2	1.2	1.1
Gold	40.2	11.3	4.9	1.9	1.1
Avg Gold	39.1	10.2	5.0	1.9	1.1
All Gold	74.5	22.3	10.5	3.0	1.7

Conclusion

- The bottleneck is not raw intelligence or context length, it is the ability to intelligently navigate, locate, and coordinate changes inside massive, noisy codebases.
- Progress on SWE-bench will require new architectures that combine long-context reasoning, active retrieval, and agent-like iteration.
- This benchmark will keep pushing the frontier of what language models can truly do in the real world.

References

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., & Narasimhan, K. (2023, October 10). SWE-Bench: Can language models resolve Real-World GitHub Issues? arXiv.org. https://arxiv.org/abs/2310.06770

Questions?

Voyager: An Open-Ended Embodied Agent with Large Language Models

arXiv:2305.16291

Feliciann Elliot

MAI 5301 - Foundations Of Large Language Models

Abstract

We introduce VOYAGER, the first LLM-powered embodied lifelong learning agent in Minecraft that continuously explores the world, acquires diverse skills, and makes novel discoveries without human intervention. VOYAGER consists of three key components: 1) an automatic curriculum that maximizes exploration, 2) an ever-growing skill library of executable code for storing and retrieving complex behaviors, and 3) a new iterative prompting mechanism that incorporates environment feedback, execution errors, and self-verification for program improvement. VOYAGER interacts with GPT-4 via black box queries, which bypasses the need for model parameter fine-tuning. The skills developed by VOYAGER are temporally extended, interpretable, and compositional, which compounds the agent's abilities rapidly and alleviates catastrophic forgetting. Empirically, VOYAGER shows strong in-context lifelong learning capability and exhibits exceptional proficiency in playing Minecraft. It obtains 3.3× more unique items, travels 2.3× longer distances, and unlocks key tech tree milestones up to 15.3× faster than prior SOTA. VOYAGER is able to utilize the learned skill library in a new Minecraft world to solve novel tasks from scratch, while other techniques struggle to generalize.

Introduction

Building embodied agents that can continuously explore and learn new skills in open-ended environments remains a major challenge in AI. Traditional approaches such as reinforcement learning and imitation learning rely on low-level actions, limiting exploration and generalization. Recent LLM-based agents can generate action plans using pre trained knowledge, but they still lack lifelong learning, meaning they cannot accumulate and reuse knowledge over time. To address this, the authors introduce VOYAGER, an LLM-powered lifelong learning agent designed for the open-ended Minecraft environment.

VOYAGER uses an automatic curriculum to propose tasks, a skill library to store reusable behaviors, and an iterative prompting mechanism that generates and refines executable code for actions using GPT-4. The agent gradually builds a library of reusable programs that can be combined to solve more complex tasks. Experiments show that VOYAGER significantly outperforms other LLM-based agents, collecting 3.3× more unique items, reaching milestones up to 15.3× faster, and exploring 2.3× farther, while also transferring learned skills to new environments.

What Problem Is This Paper Solving?

SWE-bench demonstrated that LLMs acting as passive coders fail on real-world tasks. The problem is architecture, not intelligence. Models lack:

- Any mechanism to explore an environment progressively
- Any memory of what worked before
- Any feedback loop to detect and correct their own errors

VOYAGER was developed in the Minecraft domain to directly tackle these gaps through a three-component architecture. Minecraft is chosen deliberately since it has no predefined end goal, requires unlocking a tech tree through exploration, and demands composing increasingly complex skills, making it an ideal testbed for lifelong learning.

What VOYAGER Is

VOYAGER is the first LLM-powered embodied lifelong learning agent. It continuously explores, acquires skills, and makes discoveries without human intervention.

Three core components, each addressing one of the failure modes identified in SWE-bench:

- Automatic Curriculum - Solves the navigation/exploration problem
- Skill Library - Solves the knowledge accumulation/reuse problem
- Iterative Prompting Mechanism - Solves the verification and self-correction problem

What VOYAGER Is

All components interact with GPT-4 via black box prompting no fine-tuning required.

It demonstrates that the right scaffolding around a capable LLM can achieve lifelong learning without touching model weights.

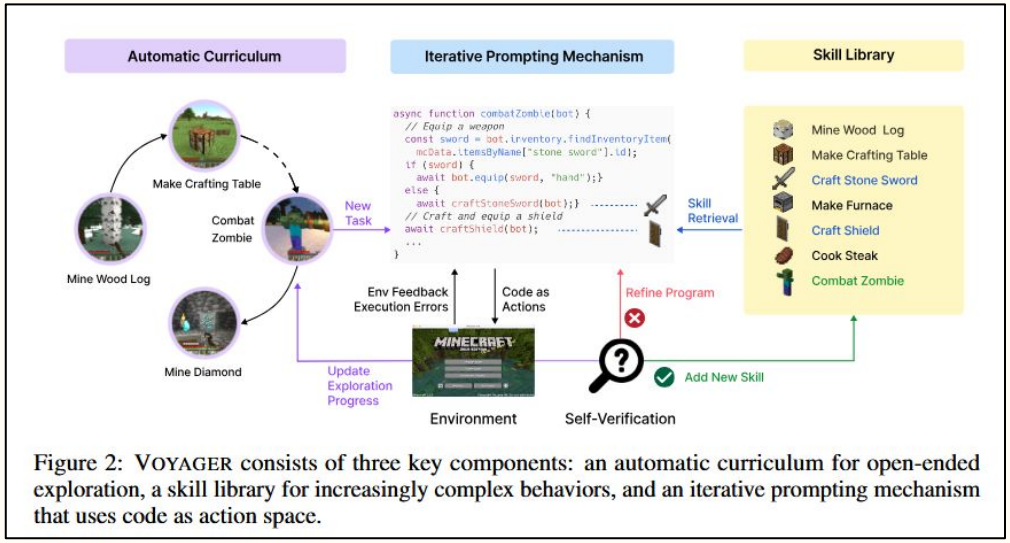

Figure 2: VOYAGER consists of three key components: an automatic curriculum for open-ended exploration, a skill library for increasingly complex behaviors, and an iterative prompting mechanism that uses code as action space.

The Components of VOYAGER

VOYAGER consists of three novel components:

(1) An automatic curriculum that suggests objectives for open-ended exploration.

(2) A skill library for developing increasingly complex behaviors, and

(3) An iterative prompting mechanism that generates executable code for embodied control.

Component 1: Automatic Curriculum

The curriculum answers: what should the agent try to do next?

- GPT-4 is prompted to propose the next task given the agent's current state (inventory, biome, time, health, nearby entities/blocks, position) and exploration history (completed + failed tasks)
- The overarching directive is "discover as many diverse things as possible", an in-context form of novelty search
- Curriculum unfolds bottom-up: starts with basic tasks (mine wood) and naturally progresses to complex ones (mine diamond) based on what the agent has mastered

Component 1: Automatic Curriculum

GPT-4 proposes the next task dynamically based on the agent's current state and exploration progress.

It receives:

- Inventory, biome, nearby entities, health/hunger, time of day
- Previously completed & failed tasks
- The Instruction: "**Discover as many diverse things as possible... but don't propose tasks that are too hard yet**"

The result is a steady stream of suitable, progressively harder tasks that keep the agent exploring without getting stuck.

Component 1: Automatic Curriculum

Why this matters compared to alternatives:

- A random curriculum causes a 93% drop in item discovery. Certain tasks attempted out of order are simply too hard without prerequisite skills
- A manual curriculum requires expert domain knowledge and can't adapt to the agent's live situation (e.g., being in a desert vs. a forest changes what's achievable)
- The automatic curriculum is adaptive, scalable, and domain-agnostic

Component 1: Automatic Curriculum

Figure 3: Tasks proposed by the automatic curriculum. We only display the partial prompt for brevity. See Appendix, Sec. A.3 for the full prompt structure.

Component 2: Skill Library

The skill library answers “How does the agent avoid forgetting what it has learned?”

- Each skill is stored as executable JavaScript code (using Mineflayer APIs) indexed by the GPT-3.5 embedding of its description
- When facing a new task, the top-5 most relevant skills are retrieved via embedding similarity and injected into GPT-4's prompt as context
- Complex skills are built by composing simpler ones. e.g., **smeltFiveRawIron()** internally calls **craftFurnace()** and **placeItem()**

Component 2: Skill Library

Why code as the representation for skills?

- Programs are temporally extended — A single function call can encode many sequential actions
- Programs are interpretable — You can read and verify what a skill does
- Programs are compositional — New skills reuse existing primitives, compounding capability over time

This directly alleviates catastrophic forgetting seen in gradient-based continual learning methods

Component 2: Skill Library

The skill library diagram showing the key-value vector database, skill retrieval query flow, and the top-5 relevant skills being returned.

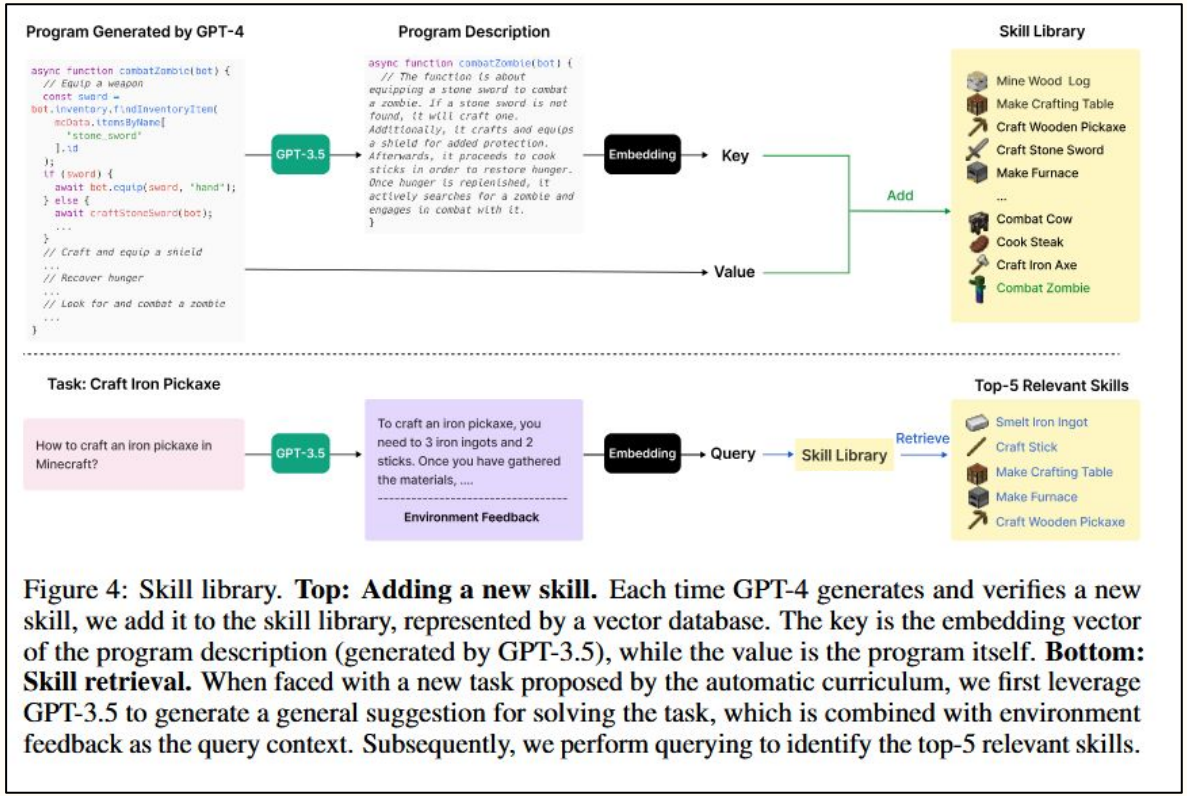

Figure 4: Skill library. **Top: Adding a new skill.** Each time GPT-4 generates and verifies a new skill, we add it to the skill library, represented by a vector database. The key is the embedding vector of the program description (generated by GPT-3.5), while the value is the program itself. **Bottom: Skill retrieval.** When faced with a new task proposed by the automatic curriculum, we first leverage GPT-3.5 to generate a general suggestion for solving the task, which is combined with environment feedback as the query context. Subsequently, we perform querying to identify the top-5 relevant skills.

Component 3: Iterative Prompting Mechanism

The iterative prompting mechanism answers: how does the agent know when it has succeeded and how does it fix failures?

Three types of feedback are incorporated into each code generation loop:

- Environment feedback — Intermediate execution state reported via bot.chat() (e.g., "I cannot make an iron chestplate because I need: 7 more iron ingots"). This tells GPT-4 why something failed.
- Execution errors — Stack traces from the JavaScript interpreter catch syntax errors and invalid API calls (e.g., attempting to craft an "acacia axe" that doesn't exist in Minecraft)
- Self-verification — A separate GPT-4 instance acts as a critic, checking whether the task was actually completed based on the agent's final state and providing a critique if not

Component 3: Iterative Prompting Mechanism

- The loop runs for up to 4 rounds per task before moving on.
  Self-verification is the most impactful component. Removing it causes a 73% drop in item discovery, because without it the agent has no reliable signal for when to commit a skill vs. when to keep trying.

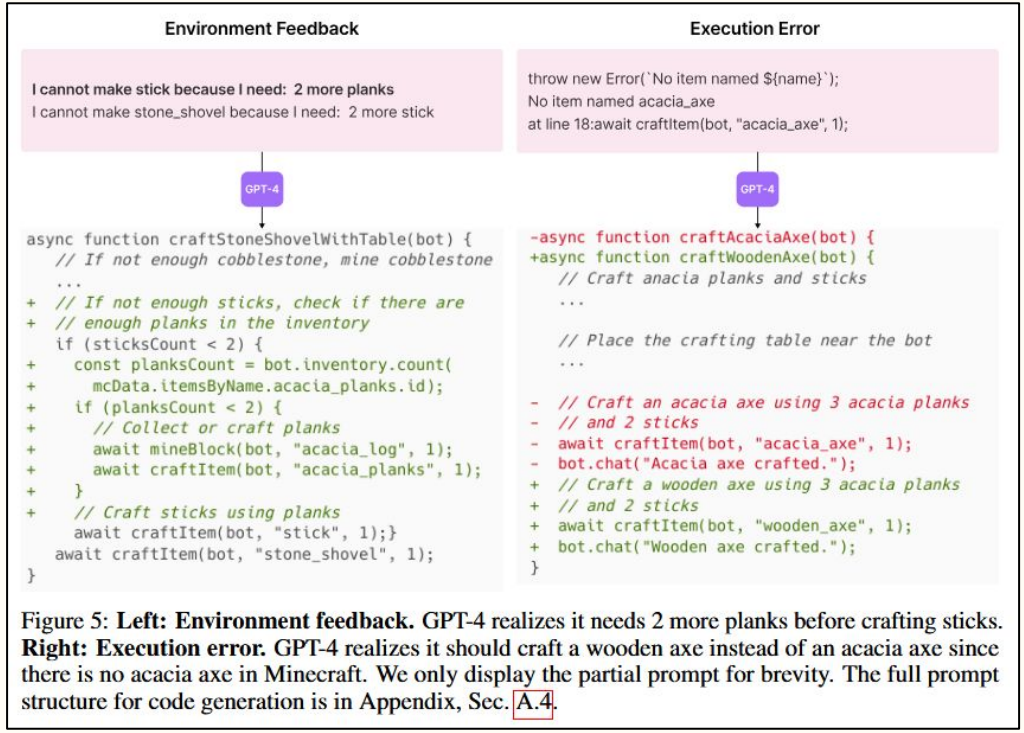

Figure 5: **Left: Environment feedback.** GPT-4 realizes it needs 2 more planks before crafting sticks. **Right: Execution error.** GPT-4 realizes it should craft a wooden axe instead of an acacia axe since there is no acacia axe in Minecraft. We only display the partial prompt for brevity. The full prompt structure for code generation is in Appendix, Sec. A.4.

Component 3: Iterative Prompting Mechanism

Diagram of the self-verification examples showing GPT-4 reasoning through success/failure with critiques.

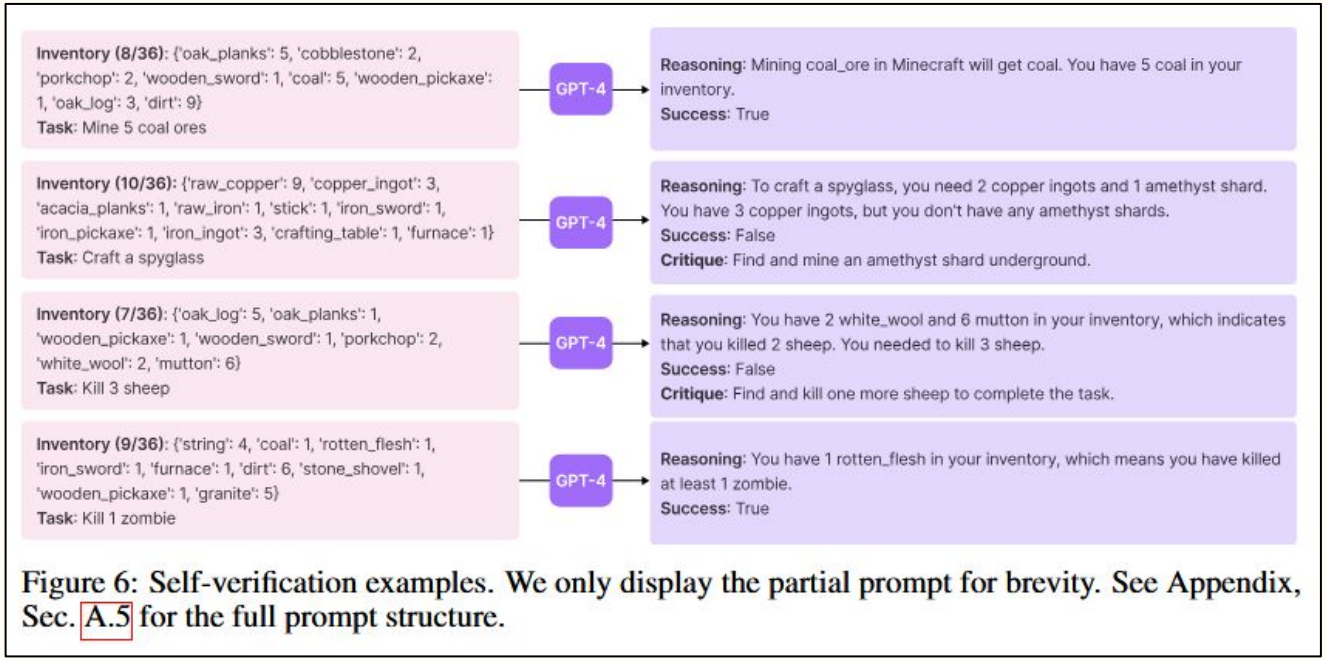

Figure 6: Self-verification examples. We only display the partial prompt for brevity. See Appendix, Sec. A.5 for the full prompt structure.

The Results

VOYAGER discovers 63 unique items within 160 prompting iterations, 3.3 times more than the next best baseline (AutoGPT). ReAct and Reflexion barely progress at all.

- On the Minecraft tech tree (wooden → stone → iron → diamond tools):
- VOYAGER unlocks the wooden level 15.3 times faster than AutoGPT
- Unlocks iron 6.4 times faster
- VOYAGER is the only method to reach the diamond level at all

The Results

The curriculum ensures tasks are always at the right difficulty level, the skill library means the agent never has to relearn how to mine wood when attempting to smelt iron, and the iterative loop ensures skills are correct before being committed to memory.

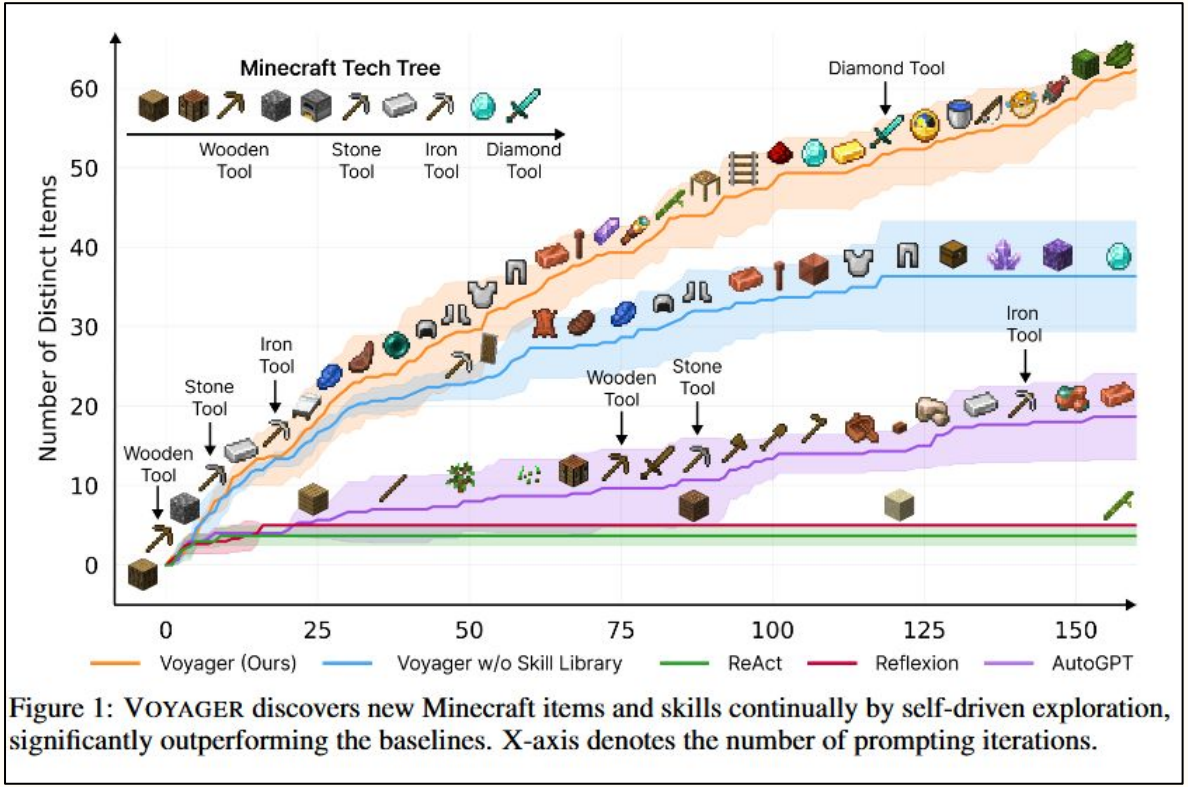

Figure 1: VOYAGER discovers new Minecraft items and skills continually by self-driven exploration, significantly outperforming the baselines. X-axis denotes the number of prompting iterations.

The Results

The tech tree mastery results showing fractions of successful trials and average prompting iterations per method.

Table 1: Tech tree mastery. Fractions indicate the number of successful trials out of three total runs. 0/3 means the method fails to unlock a level of the tech tree within the maximal prompting iterations (160). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Wooden Tool	Stone Tool	Iron Tool	Diamond Tool
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	92 ± 72 (3/3)	94 ± 72 (3/3)	135 ± 103 (3/3)	N/A (0/3)
VOYAGER w/o Skill Library	7 ± 2 (3/3)	9 ± 4 (3/3)	29 ± 11 (3/3)	N/A (0/3)
VOYAGER (Ours)	6 ± 2 (3/3)	11 ± 2 (3/3)	21 ± 7 (3/3)	102 (1/3)

Ablation Studies

- Without automatic curriculum there is a 93% drop in items as random task ordering breaks the learning process entirely
- Without skill library the agent plateaus in later stages since it cannot build on prior work, so complexity never compounds
- GPT-4 gets 5.7 times more unique items while the quality of the code-generating backbone is non-negotiable

Ablation Studies

- Without self-verification there is a 73% drop as the agent can't distinguish success from failure, so it commits broken skills and makes no real progress
- Without execution errors there is a meaningful drop. Syntax-level feedback is necessary for the code refinement loop to converge

Ablation Studies

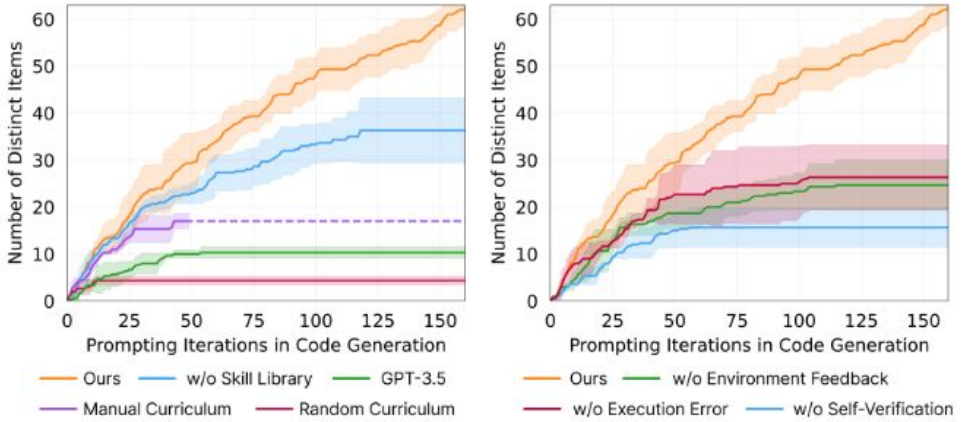

Figure 9: **Left: Ablation studies for the automatic curriculum, skill library, and GPT-4.** GPT-3.5 means replacing GPT-4 with GPT-3.5 for code generation. VOYAGER outperforms all the alternatives, demonstrating the critical role of each component. **Right: Ablation studies for the iterative prompting mechanism.** VOYAGER surpasses all the other options, thereby highlighting the essential significance of each type of feedback in the iterative prompting mechanism.

Zero-Shot Generalization

After training in one Minecraft world, VOYAGER is moved to a completely new world with a cleared inventory and asked to solve novel tasks (diamond pickaxe, golden sword, lava bucket, compass).

- VOYAGER solves all four tasks consistently
- ReAct, Reflexion, and AutoGPT solve none
- Even AutoGPT with VOYAGER's skill library plugged in improves substantially, demonstrating the skill library is a transferable, plug-and-play asset, not specific to VOYAGER's other components

Zero-Shot Generalization

Table 2: Zero-shot generalization to unseen tasks. Fractions indicate the number of successful trials out of three total attempts. 0/3 means the method fails to solve the task within the maximal prompting iterations (50). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Diamond Pickaxe	Golden Sword	Lava Bucket	Compass
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28] w/ Our Skill Library	39 (1/3)	30 (1/3)	N/A (0/3)	30 (2/3)
VOYAGER w/o Skill Library	36 (2/3)	30 ± 9 (3/3)	27 ± 9 (3/3)	26 ± 3 (3/3)
VOYAGER (Ours)	$\mathbf{19 \pm 3}$ (3/3)	$\mathbf{18 \pm 7}$ (3/3)	$\mathbf{21 \pm 5}$ (3/3)	$\mathbf{18 \pm 2}$ (3/3)

SWE-bench vs VOYAGER

SWE-bench and VOYAGER converge on the same architectural conclusions from opposite directions:

SWE-bench's Shortcomings	VOYAGER's Architectural Solution
No environment navigation	Automatic curriculum with agent-state awareness
No knowledge accumulation	Skill library with embedding-based retrieval
No self-correction	Iterative prompting with 3 feedback types + self-verification
Context overload	Code as actions — compact, composable, interpretable

The Overall Limitations

SWE-bench limitations:

- All tasks are done in Python. There should be support for other languages
- Evaluation is execution-based but can't guarantee code quality, efficiency, or readability
- Retrieval remains a major unsolved bottleneck

VOYAGER limitations:

- GPT-4 API costs are substantial (15 times more expensive than GPT-3.5)
- Curriculum sometimes proposes non-existent items ("copper sword"), code generation sometimes calls functions that don't exist
- There is no visual perception and the solution relies entirely on text-based environment feedback

Conclusion

- The same class of model GPT-4 becomes a genuinely capable lifelong learner when wrapped in the right scaffolding. The curriculum keeps tasks achievable. The skill library means progress compounds rather than resets.
- The iterative feedback loop means the agent always knows whether it has succeeded. None of these components are individually significant, but together they produce behavior that no baseline comes close to matching.
- The frontier of LLM capability is not about making models smarter in isolation. It's about designing systems that let existing models explore intelligently, remember what they learn, and correct themselves when they are wrong.

References

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023, May 25). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv.org. https://arxiv.org/abs/2305.16291

Questions?